Διάλεξη 9 - Δεδομένα Εισόδου

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Εισαγωγή στον Προγραμματισμό

Θανάσης Αυγερινός / Τάκης Σταματόπουλος

Ανακοινώσεις / Διευκρινίσεις

- Χρήση τελεστή ισότητας με ορίσματα float / double
- Ονομασία συναρτήσεων camelCase vs snake_case

1η Θέση στο GRCPC! (+ Αργυρό + 3 θέσεις στην 11άδα)

RANK	TEAM	SCORE	A	B	C	D	E	F	G	H	I	J	K	L	M
1	**?DIcomposition** National and Kapodistrian University of Athens	8 1169		296 10 tries	259 1 try	77 1 try	28 2 tries	1 try	204 2 tries	1 try	11 1 try	31 3 tries		1 try	3 1 try
2	**PAPAPAP** National Technical University of Athens	7 602				49 3 tries	57 1 try		143 3 tries		15 1 try	31 1 try		218 1 try	9 1 try
3	**Segment Three** National Technical University of Athens	7 710		11 tries		54 2 tries	58 1 try		143 4 tries		10 2 tries	20 2 tries		243 4 tries	2 1 try
4	**Wrong answer on test 2** National Technical University of Athens	6 371		3 tries	2 tries	22 1 try	40 1 try		218 1 try		15 1 try	55 2 tries			1 1 try
5	**omada pou xanei den kerdizei** National and Kapodistrian University of Athens	6 488		4 tries		125 1 try	34 1 try		223 4 tries		18 1 try	24 1 try		1 try	4 1 try
6	**CompilerOfProblems** Aristotle University of Thessaloniki	6 902	2 tries			68 1 try	297 3 tries	1 try	290 1 try		77 1 try	106 2 tries			4 1 try
7	**CLaplace** University of Patras	5 332		2 tries		140 1 try	74 2 tries				31 1 try	48 1 try			19 1 try
8	**SegTree Climbers** National Technical University of Athens	5 481				96 2 tries	147 1 try				53 1 try	119 2 tries			26 1 try
9	**Sort and pray** Athens University of Economics and Business	5 484				222 1 try	72 1 try		6 tries		38 1 try	110 2 tries			22 1 try
10	**VarvTeamReps** National Technical University of Athens	5 579				189 1 try	116 4 tries	1 try			33 1 try	96 5 tries	3 tries		5 1 try
11	**RBG** National and Kapodistrian University of Athens	5 581				135 1 try	121 2 tries		5 tries		48 2 tries	152 5 tries			5 1 try

eam/61/problem/grcpc2026-598832

https://grcpc.gr/public?countries%5B%5D=GRC&filter=filter

Τι θα τυπώσει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
    float a = 3.1;

    if(a == 3.1)
       printf("OK\n");
    else
       printf("Not OK\n");
    return 0;
}
```

Υπάρχει κάποια εξήγηση;

Τι θα τυπώσει το παρακάτω πρόγραμμα;

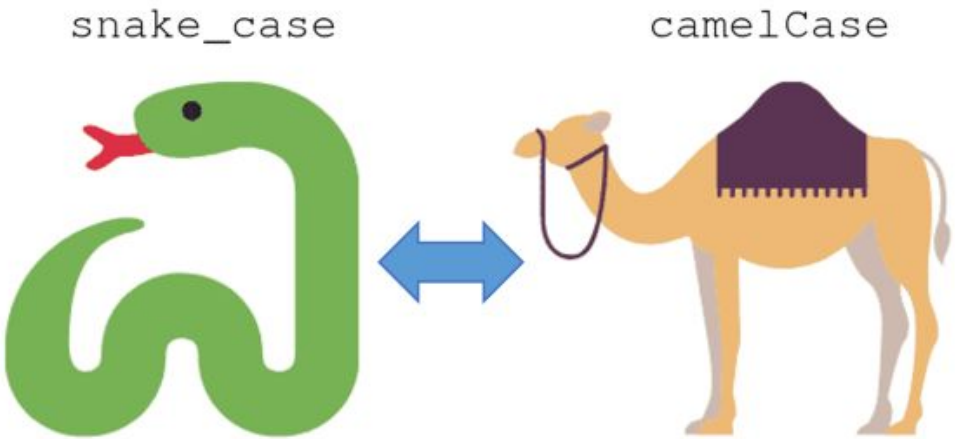
```c
#include <stdio.h>
int main() {
    float a = 3.1;

    if(a == 3.1)
       printf("OK\n");
    else
       printf("Not OK\n");
    return 0;
}
```

```
$ ./one
Not OK
```

Υπάρχει κάποια εξήγηση;

camelCase vs snake_case

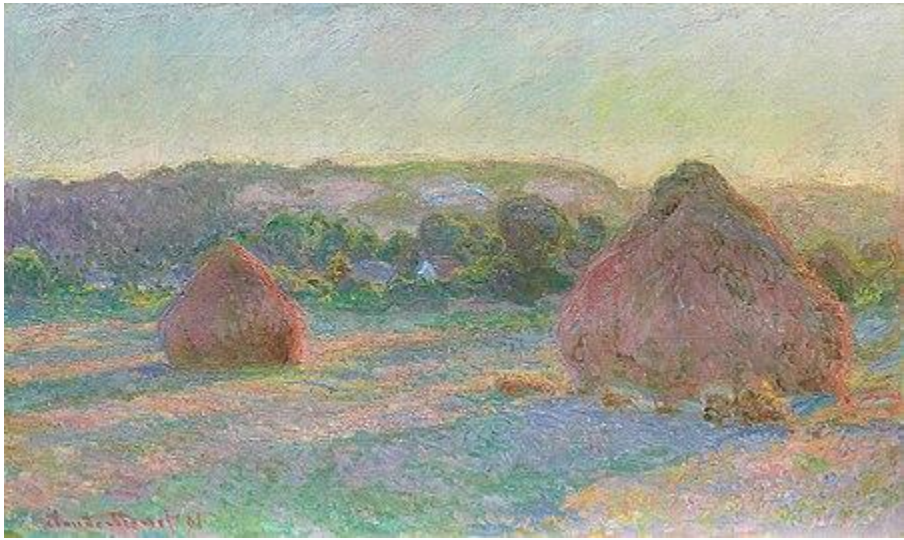


Για κώδικα C/Python συνιστώ `snake_case`

Παράδειγμα: `pi_approx` είναι καλύτερο (υποκειμενικό) από `piApprox`

Την Προηγούμενη Φορά

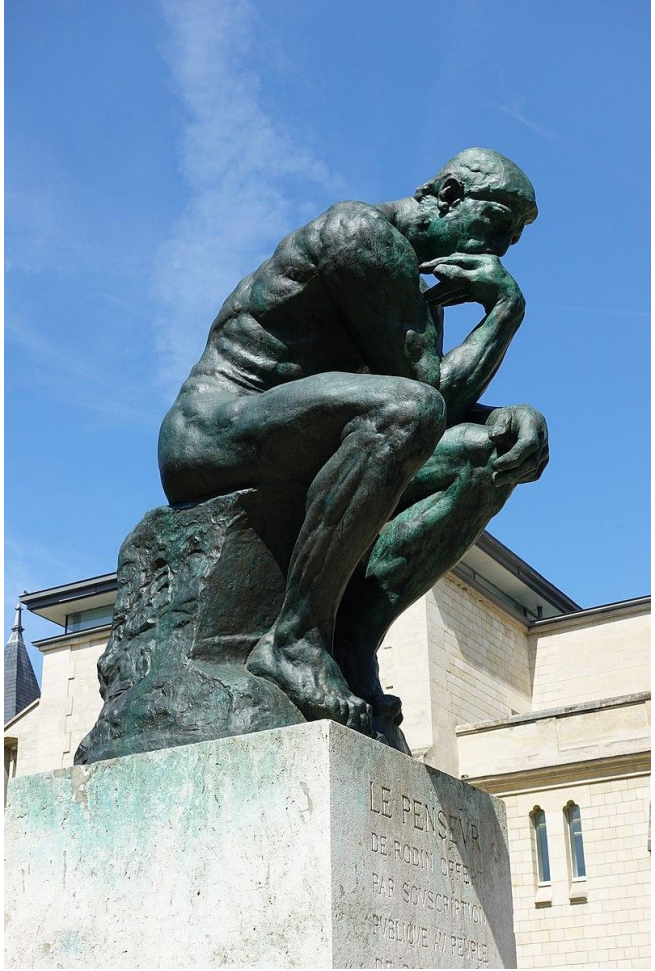
- Παραδείγματα Επαναληπτικών Δομών
- Άλλες Δομές Ελέγχου
- Επίλυση Προβλημάτων



Haystacks (End of Summer) by Claude Monet [1891]

Σήμερα

- Δεδομένα Εισόδου και εξόδου
- Παραδείγματα



The Thinker by Auguste Rodin [1904]

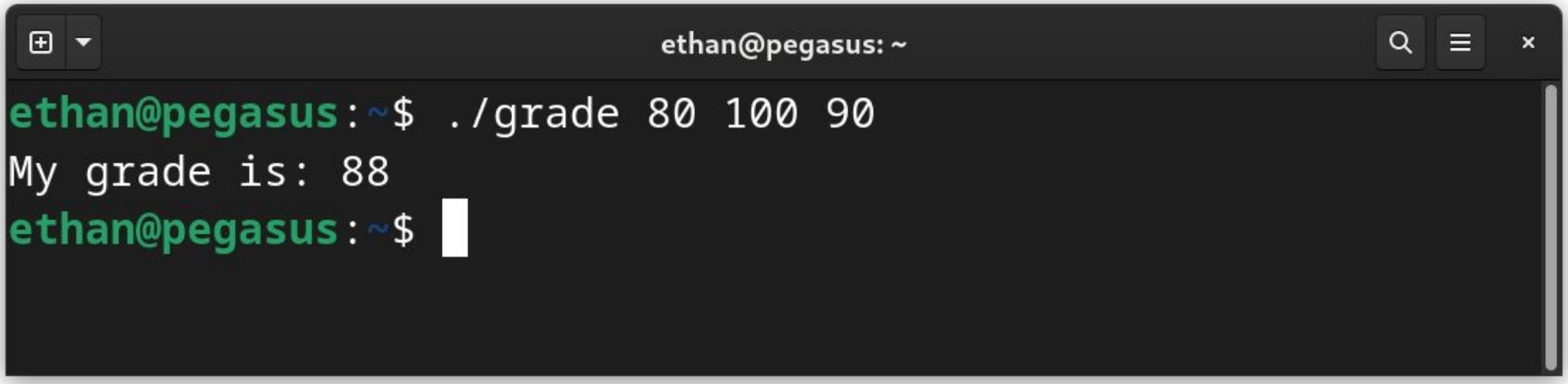
Δεδομένα Εισόδου και Εξόδου (Input and Output Data)



Δεδομένα Εισόδου (Input Data)

Τα δεδομένα εισόδου (input data) είναι μια σειρά από χαρακτήρες (bytes) τα οποία ο χρήστης δίνει στο πρόγραμμα. Υπάρχουν 4 μέθοδοι να εισάγουμε δεδομένα:

1. Ορίσματα στην γραμμή εντολών



Δεδομένα Εισόδου (Input Data) - 2/4

Τα δεδομένα εισόδου (input data) είναι μια σειρά από χαρακτήρες (bytes) τα οποία ο χρήστης δίνει στο πρόγραμμα. Υπάρχουν 4 μέθοδοι να εισάγουμε δεδομένα:

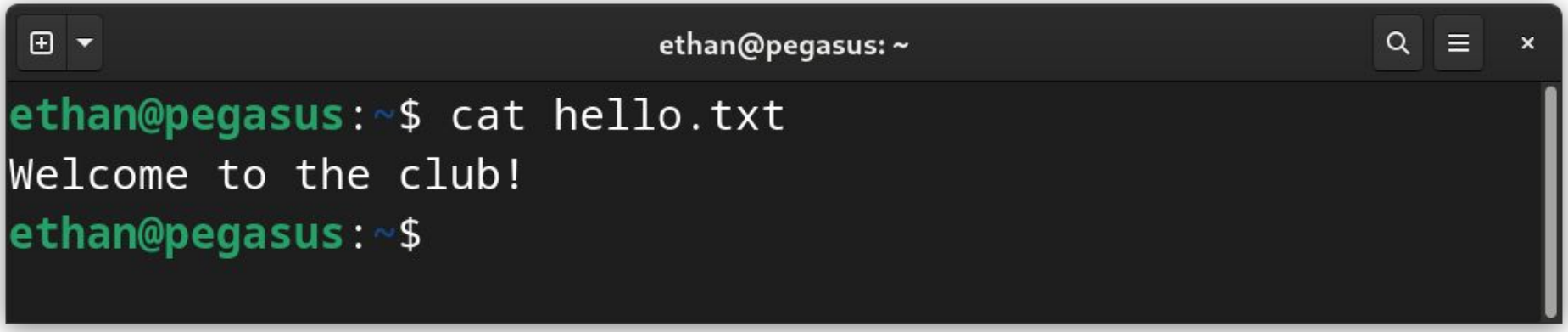
2. Γράφοντας κείμενο στην πρότυπη είσοδο (standard input ή stdin) συνήθως με το πληκτρολόγιο

```
thanassis@Plato:~$ ./goldbach
Give me a number: 42
Choose how to test 42 - ('p' for prime, 'g' for goldbach) g
42 = 5 + 37
Give me a number: EOF detected, exiting
thanassis@Plato:~$
```

Δεδομένα Εισόδου (Input Data) - 3/4

Τα δεδομένα εισόδου (input data) είναι μια σειρά από χαρακτήρες (bytes) τα οποία ο χρήστης δίνει στο πρόγραμμα. Υπάρχουν 4 μέθοδοι να εισάγουμε δεδομένα:

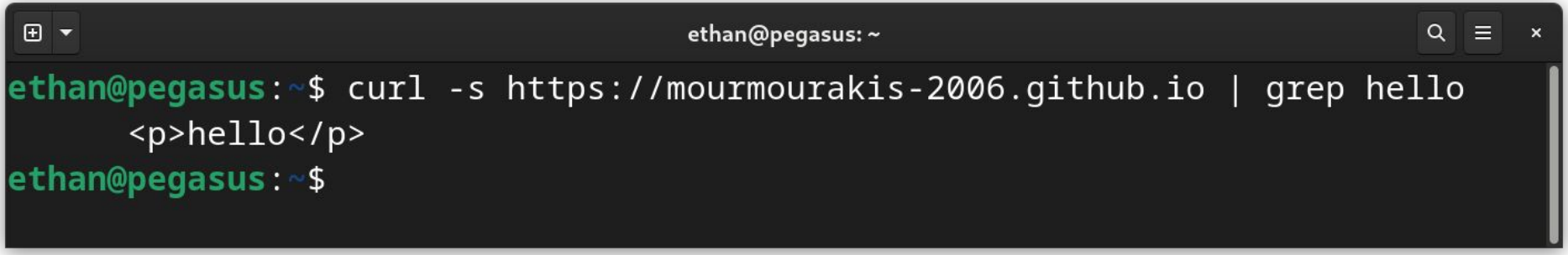
3. Διαβάζοντας αρχεία από το σύστημα αρχείων (επόμενες διαλέξεις)



Δεδομένα Εισόδου (Input Data) - 4/4

Τα δεδομένα εισόδου (input data) είναι μια σειρά από χαρακτήρες (bytes) τα οποία ο χρήστης δίνει στο πρόγραμμα. Υπάρχουν 4 μέθοδοι να εισάγουμε δεδομένα:

4. Διαβάζοντας από το δίκτυο ή άλλες πηγές - π.χ., User Interface (σε επόμενα εξάμηνα)



Δεδομένα Εισόδου (Input Data)

Τα δεδομένα εισόδου (input data) είναι μια σειρά από χαρακτήρες (bytes) τα οποία ο χρήστης δίνει στο πρόγραμμα. Υπάρχουν 4 μέθοδοι να εισάγουμε δεδομένα:

1. Ορίσματα στην γραμμή εντολών ✅
2. Γράφοντας κείμενο στην πρότυπη είσοδο (standard input ή stdin) ✅
3. Διαβάζοντας αρχεία από το σύστημα αρχείων (επόμενες διαλέξεις)
4. Διαβάζοντας από το δίκτυο ή άλλες πηγές (άλλα εξάμηνα)



Η συνάρτηση `getchar()`

Η συνάρτηση `getchar()` ορίζεται στο stdio.h, διαβάζει έναν χαρακτήρα *εάν υπάρχει* από το stdin του προγράμματος και τον επιστρέφει ως ακέραιο. Αν δεν υπάρχει, επιστρέφει την τιμή End-Of-File / EOF (-1).

Η συνάρτηση getchar() είναι "έτοιμη" για χρήση από μας από το stdio.h - πως μπορώ να βρω πως συμπεριφέρεται;

Ανοίγω ένα τερματικό και τρέχω `man getchar`!

Η συνάρτηση `getchar()`

Η συνάρτηση `getchar()` ορίζεται στο stdio.h, διαβάζει έναν χαρακτήρα *εάν υπάρχει* από το stdin του προγράμματος και τον επιστρέφει ως ακέραιο. Αν δεν υπάρχει, επιστρέφει την τιμή End-Of-File / EOF (-1). Η συνάρτηση έχει την ακόλουθη μορφή:

```
int getchar();
```

Δεν παίρνει κανένα όρισμα και επιστρέφει έναν ακέραιο.

Χρήση της συνάρτησης `getchar()`

Για να διαβάσουμε έναν χαρακτήρα και να τον τυπώσουμε, γράφουμε:

```c
#include <stdio.h>
int main() {
  printf("Gimme a char: ");
  int ch = getchar();
  if (ch != EOF) {
     printf("You gave the char: %c\n", ch);
  } else {
     printf("input ended\n");
  }
  return 0;
}
```

Χρήση της συνάρτησης `getchar()`

Για να διαβάσουμε έναν χαρακτήρα και να τον τυπώσουμε, γράφουμε:

```c
#include <stdio.h>
int main() {
  printf("Gimme a char: ");
  int ch = getchar();
  if (ch != EOF) {
     printf("You gave the char: %c\n", ch);
  } else {
     printf("input ended\n");
  }
  return 0;
}
```

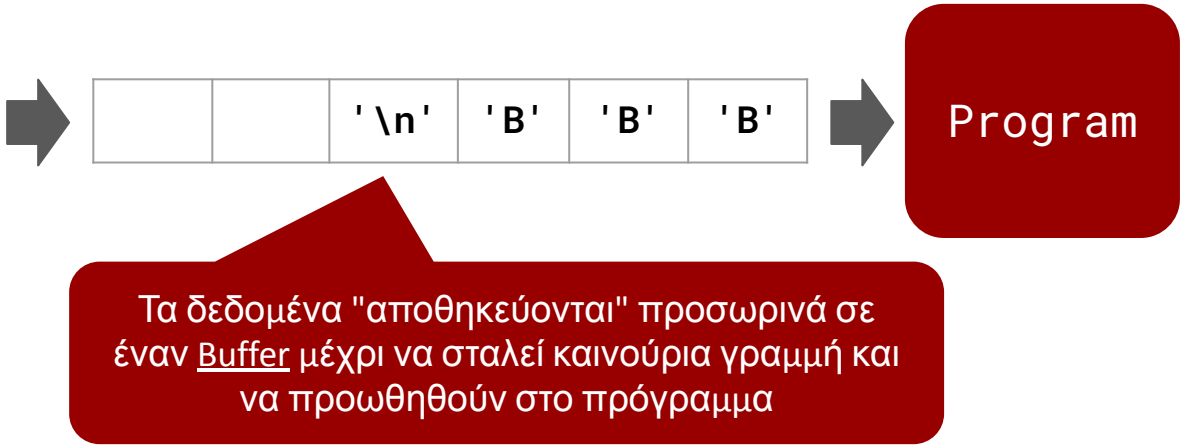
Η getchar θα διαβάσει μόνο έναν χαρακτήρα

```
$ ./chartest
Gimme a char: BBB
You gave the char: B
```

Χρήση της συνάρτησης `getchar()`

Συνήθως, πρέπει να περιμένουμε μέχρι να πατήσουμε Enter προκειμένου οι χαρακτήρες που πληκτρολογήσαμε να φτάσουν το πρόγραμμα.

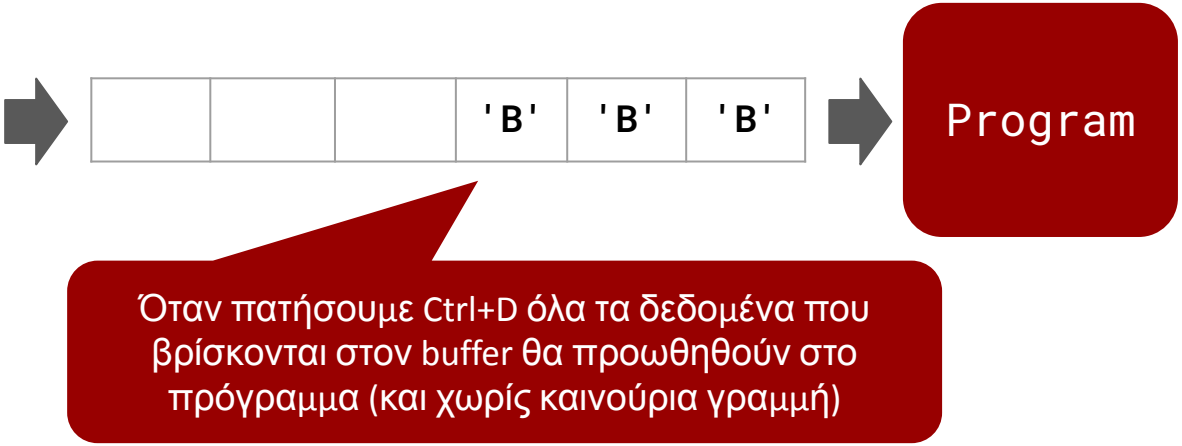




Χρήση της συνάρτησης `getchar()`

Για να δείξουμε ότι τελείωσαν τα δεδομένα εισόδου, στο Linux συνήθως πρέπει να πατήσουμε Ctrl+D (EOF)





Χρήση της συνάρτησης getchar()

Διαδοχικές κλήσεις της getchar() διαβάζουν διαδοχικούς χαρακτήρες. Τι κάνει το παρακάτω πρόγραμμα;

```
#include <stdio.h>
int main() {
  int ch, sum = 0;
  printf("Enter characters: ");
  while( (   ch = getchar()   ) != EOF ) {
     printf("%c", ch);
     sum++;
  }
  printf("\nTotal characters: %d\n", sum);
  return 0;
}
```

Χρήση της συνάρτησης `getchar()`

Διαδοχικές κλήσεις της getchar() διαβάζουν διαδοχικούς χαρακτήρες. Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>

int main() {

  int ch, sum = 0;

  printf("Enter characters: ");

  while( (ch = getchar()) != EOF ) {

      printf("%c", ch);

      sum++;

  }

  printf("\nTotal characters: %d\n", sum);

  return 0;

}
```

```
$ ./charcount
Enter characters: we'll always have paris
we'll always have paris
Total characters: 24
```

Η συνάρτηση `putchar()`

Η συνάρτηση `putchar()` ορίζεται στο stdio.h, παίρνει έναν χαρακτήρα ως όρισμα, τον τυπώνει στο stdout του προγράμματος και τον επιστρέφει ως ακέραιο. Αν κάτι δεν πάει καλά στο τύπωμα, επιστρέφει την τιμή EOF (-1). Η συνάρτηση έχει την ακόλουθη μορφή:

```
int putchar(int c);
```

Προκειμένου να τυπώσουμε έναν χαρακτήρα `'C'` απλά γράφουμε `putchar('C');`. Η συνάρτηση αυτή είναι είναι ένα απλούστερο υποσύνολο της `printf`.

Τι πρόβλημα έχει η παρακάτω υλοποίηση της `cat`

```c
#include <stdio.h>

int main() {
  char c;
  while((c = getchar()) != EOF)
    putchar(c);
  return 0;
}
```

Τι πρόβλημα έχει η παρακάτω υλοποίηση της `cat`

```c
#include <stdio.h>


int main() {
  char c;
  while((c = getchar()) != EOF)
    putchar(c);
  return 0;
}
```

```
$ echo -e "hello\xffworld" | ./cat
hello
$ echo -e "hello\xffworld" | cat
helloworld
```

Προσοχή! Πρέπει να οριστεί ως **int**, όχι **char**

Προσοχή: πάντα αναθέτουμε την τιμή επιστροφής της getchar() εκτός και αν είμαστε σίγουροι για το τι κάνουμε

Θέλω να διαβάσω δύο αριθμούς από την πρότυπη είσοδο και να τους προσθέσω. Πως;

```
$ ./addnums
Give me a number: 40
Give me another number: 2
Total: 42
```

Θέλω να διαβάσω δύο αριθμούς από την πρότυπη είσοδο και να τους προσθέσω. Πως;

```
$ ./addnums
Give me a number: 40
Give me another number: 2
Total: 42
```

Κάνοντας χρήση της `getchar` και φτιάχνοντας μια συνάρτηση getinteger προκειμένου να διαβάσουμε τους χαρακτήρες έναν-έναν και να τους μετατρέψουμε σε αριθμό. Υπάρχει άλλος τρόπος να επιτύχουμε το ίδιο αποτέλεσμα;

```c
#define ERROR -1

int getinteger(int base) {
  int ch;
  int val = 0;
  while ((ch = getchar()) != '\n')
    if (ch >= '0' && ch <= '0' + base - 1)
      val = base * val + (ch - '0');
    else
      return ERROR;
  return val;
}
```

ACT I — Setup: the gun appears



a promise to the audience

ACT III — Payoff: the gun is fired



Chekhov’s Gun

Every element in a story
should earn its place.
If a rifle hangs on the wall
in the first act, it must
go off by the last.

— dramatic principle of Anton Chekhov

Η συνάρτηση scanf

Η συνάρτηση scanf ορίζεται στο header file stdio.h και χρησιμοποιείται για να διαβάζει δεδομένα εισόδου πολλών τύπων από το stdin του προγράμματος και να αποθηκεύσει τις τιμές τους σε μεταβλητές. Αν επιτύχει, επιστρέφει πόσα δεδομένα εισόδου διάβασε. Αν αποτύχει, επιστρέφει την τιμή End-Of-File / EOF (-1).

Πως μπορώ να βρω πως συμπεριφέρεται;

Ανοίγω ένα τερματικό και τρέχω
`man scanf`!

Η συνάρτηση scanf

Η συνάρτηση scanf ορίζεται στο header file stdio.h και χρησιμοποιείται για να διαβάζει δεδομένα εισόδου πολλών τύπων από το stdin του προγράμματος και να αποθηκεύσει τις τιμές τους σε μεταβλητές. Αν επιτύχει, επιστρέφει *πόσα δεδομένα εισόδου* διάβασε. Αν αποτύχει, επιστρέφει την τιμή End-Of-File / EOF (-1). Η συνάρτηση έχει την ακόλουθη μορφή:

```
int scanf(const char *restrict format, ...);
```

Έχει μια συμβολοσειρά μορφοποίησης (format string)

Δέχεται όσα ορίσματα περάσουμε (άλλο μάθημα)

Η συνάρτηση scanf

Η συνάρτηση scanf ορίζεται στο header file stdio.h και χρησιμοποιείται για να διαβάζει δεδομένα εισόδου πολλών τύπων από το stdin του προγράμματος και να αποθηκεύσει τις τιμές τους σε μεταβλητές. Αν επιτύχει, επιστρέφει πόσα δεδομένα εισόδου διάβασε. Αν αποτύχει, επιστρέφει την τιμή End-Of-File / EOF (-1). Η συνάρτηση έχει την ακόλουθη μορφή:

```
int scanf(const char *restrict format, ...);
int printf(const char *restrict format, ...);
```

Είναι η συμμετρική της printf για διάβασμα αντί για εκτύπωση

Χρήση της συνάρτησης `scanf`

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n;
  printf("Gimme a number: ");
  scanf("%d", &n);
  printf("Square: %d\n", n * n);
  return 0;
}
```

Χρήση της συνάρτησης scanf

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n;
  printf("Gimme a number: ");
  scanf("%d", &n);
  printf("Square: %d\n", n * n);
  return 0;
}
```

Περνάμε την **διεύθυνση (spoiler)** της μεταβλητής n στην μνήμη ώστε η scanf να μπορέσει να αναθέσει την τιμή που διάβασε

Τυπώνει στο stdout το τετράγωνο του αριθμού που γράψαμε στο stdin

Χρήση της συνάρτησης scanf

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n;
  printf("Gimme a number: ");
  scanf("%d", &n);
  printf("Square: %d\n", n * n);
  return 0;
}
```

Τι θα γινόταν αν γράφαμε `scanf("%d", n);` Γιατί;

```
$ ./scanf
Gimme a number: 3
Segmentation fault
```

Περνάμε την **διεύθυνση (spoiler)** της μεταβλητής n στην μνήμη ώστε η scanf να μπορέσει να αναθέσει την τιμή που διάβασε

Τυπώνει στο stdout το τετράγωνο του αριθμού που γράψαμε στο stdin

Χρήση της συνάρτησης `scanf`

Τι κάνει το παρακάτω πρόγραμμα;

Είναι σωστό αυτό το πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n;
  printf("Gimme a number: ");
  scanf("%d", &n);
  printf("Square: %d\n", n * n);
  return 0;
}
```

Χρήση της συνάρτησης scanf

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n;
  printf("Gimme a number: ");
  scanf("%d", &n);
  printf("Square: %d\n", n * n);
  return 0;
}
```

Είναι σωστό αυτό το πρόγραμμα;

Όχι καθώς δεν ελέγχουμε την τιμή επιστροφής της scanf (EOF ή ίσως 0!)

Χρήση της συνάρτησης scanf

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n;
  printf("Gimme a number: ");
  scanf("%d", &n);
  printf("Square: %d\n", n * n);
  return 0;
}
```

```
$ ./scanf
Gimme a number: 16
Square: 256
$ ./scanf
Gimme a number: Square:
1068701481
$ ./scanf
Gimme a number: hello
Square: 1072038564
```

Χρήση της συνάρτησης `scanf` - Πολλά ορίσματα

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n1, n2;
  printf("Gimme two numbers: ");
  scanf("%d %d", &n1, &n2);
  printf("Result: %d\n", n1 * n2);
  return 0;
}
```

Χρήση της συνάρτησης `scanf` - Πολλά ορίσματα

Τι κάνει το παρακάτω πρόγραμμα;

```c
#include <stdio.h>
int main() {
  int n1, n2;
  printf("Gimme two numbers: ");
  scanf("%d %d", &n1, &n2);
  printf("Result: %d\n", n1 * n2);
  return 0;
}
```

```
$ ./scanf2
Gimme two numbers:   2   4
Result: 8
```

Καθώς ψάχνει για δεκαδικό ψηφίο, η scanf αγνοεί τους κενούς χαρακτήρες ή αλλαγές γραμμής

Τι κάνει το παρακάτω πρόγραμμα;

```c
#define ERROR -1                              // Return value for illegal character
int getinteger(int base) {
  int ch;                                     // No need to declare ch as int - no EOF handling
  int val = 0;                                // Initialize return value
  while ((ch = getchar()) != '\n')            // Read up to new line
      if (ch >= '0' && ch <= '0' + base - 1) // Legal character?
        val = base * val + (ch - '0');       // Update return value
      else
        return ERROR;   // Illegal character read
  return val;   // Everything OK - Return value of number read
}
```

Τι κάνει το παρακάτω πρόγραμμα;

```
  int i, ch, total = 0;
  int letfr[26];  // Letter occurrences and frequencies array
  for (i=0 ; i < 26 ; i++)
    letfr[i] = 0;
  while ((ch = getchar()) != EOF) {
    if (ch >= 'A' && ch <= 'Z') {
      letfr[ch-'A']++;             // Found upper case letter
      total++;
    }
    if (ch >= 'a' && ch <= 'z') {
      letfr[ch-'a']++;             // Found lower case letter
      total++;
    }
}
```

Η συνάρτηση `fprintf`

Η συνάρτηση `fprintf` είναι όμοια με την printf, απλά αντί να γράφει στο stdout, μπορεί να γράψει σε οποιοδήποτε αρχείο. Η συνάρτηση έχει την ακόλουθη μορφή:

```
int fprintf(FILE *stream, const char *format, ...);
```

Η κλήση printf(x, y, z) είναι ουσιαστικά ισοδύναμη με την fprintf(stdout, x, y, z)

Παράδειγμα χρήσης fprintf

```c
#include <stdio.h>
int main() {
    ...
    // how to print to standard error (stderr)
    fprintf(stderr, "Not a prime number: %d\n", num);

    return 0;
}
```

Για την επόμενη φορά

- Καλύψαμε έννοιες από τις σελίδες 28-29, 70-71, 78-79, 86-87 από τις σημειώσεις του κ. Σταματόπουλου.
- getchar , putchar, scanf, printf
- Data buffer
- End of transmission
- Canonical mode

Ευχαριστώ και καλή μέρα εύχομαι!
Keep Coding ;)